



Урок 2 > Сбор требований к системе

> Введение

Собирать требования тяжело. Со слов заказчика вы должны понять, что ему действительно нужно.

Иногда люди не видят систему комплексно.

Помимо этого, часто ошибки начинаются с путаницы из-за explicit и implicit. Когда люди договариваются друг с другом, они могут подразумевать что-то (*implicit*), полагая, что другая сторона тоже это подразумевает. В противоположном случае можно явно перечислить всё необходимое, проговорить последовательность и способы (*explicit*).

Очевидно, необходимо заранее собирать требования к системе.

Формирование требований при проектировании большой системы — важная и сложная аналитическая задача с подводными камнями, даже при самом базовом подходе к которой, будет недостаточно набросать все важные функции системы в большой список и на всякий случай потребовать, чтобы она всегда отвечала быстро, точно и была надёжной.

Скорее всего, у получившегося списка требований найдется масса проблем: неявные противоречия, неучтённые производные требования, неучтённые категории пользователей и стейкхолдеров или возможно целые непокрытые при первичном анализе технические области, в которых есть важные новые вводные, сильно меняющие картину. В идеале, процесс формулирования требований должен быть структурированным и итеративным, а сами требования должны удовлетворять ряду критериев, улучшающих их качество и точность.

Вообще работу с требованиями можно выделить как один из разделов такой дисциплины как “Системный Анализ”. Именно системные аналитики решают задачи сбора, учёта, обработки и приведения в конечную форму требований при проектировании больших систем. Конечно, совсем необязательно осваивать еще одну профессию, чтобы написать хорошие требования в рамках курса

по системному дизайну, но несколько важных приемов из этой области знаний существенно улучшат качество предположений, исходя из которых мы будем проектировать свои системы.

> Сбор требований к системе

Так как задача проектирования системы ставится в открытой форме, нам самим нужно ограничить функционал и определить требования к системе.

Приступая к любому дизайну системы, стоит начать с ответов на основные вопросы:

- Функциональные требования — какие основные возможности даёт система пользователям?
- Нефункциональные требования — насколько надёжной и отзывчивой должна быть система?
- Ограничение функционала.

Вспомним ещё раз, что в самом первом приближении, функциональные требования — это возможности, которые система явным образом предоставляет пользователю, а нефункциональные — это совокупность разного рода ограничений и необходимых количественных и качественных характеристик. Хорошим эмпирическим приёмом для того, чтобы понять, с каким требованием мы имеем дело, может быть ещё одно классическое определение: функциональные требования — это требования к системе, которые останутся необходимыми, даже если она будет работать на гипотетическом идеальном компьютере с неограниченными скоростью вычислений и памятью, полным отсутствием источников ошибок, нулевой стоимостью эксплуатации и отсутствием источников вредоносной активности.

Как вообще написать хорошее требование:

- Требование должно быть однозначно интерпретируемым и объективным. Каждое требование описывает только одну функциональность или свойство и содержит достаточно необходимых деталей;
- Требование должно быть тестируемым. На основе каждого из требований можно при необходимости написать тест-кейс, описывающий некоторую детерминированную процедуру шагов, ведущую к проверяемому результату;
- Для большинства качественных нефункциональных требований очень важным является их измеримость или другая мера соответствия, явно заложенная в базовую формулировку. Так, для большинства категорий качественных требований, существуют общепринятые метрики или стандарты, на которые следует опираться.

Чтобы не забыть о каком-то аспекте или области, со стороны которых стоит хотя бы раз поискать дополнительные важные требования к системе на этапе первичного сбора, можно в дальнейшем

выделить более специфические категории.

> Функциональные требования

Бизнес-требования — это непосредственный ключевой функционал системы, те фичи, которые получат наши конечные пользователи, для которых система создаётся.

Довольно распространённым правилом написания бизнес-требований является формат user story, следующий примерно шаблону “как <тип пользователя>, я могу использовать <функционал>, чтобы <желаемый результат>” (иначе говоря, быть [behavior-driven](#)).

Например, следующие три требования будут примерами бизнес-требований, описывающих премиум-пользователей в Твиттере:

- Как премиум-пользователь, я буду иметь синюю галочку справа от юзернейма;
- Премиум-пользователи проходят при регистрации обязательную процедуру биометрической верификации.

Пользователи могут быть разными, например, потенциальными.

- Как неавторизованный посетитель страницы x.com, я должен видеть три самых популярных твита за последний час, написанных юзерами из страны моего исходного ip-адреса.

В случае технических требований, для которых шаблон user story не применим, формат требований может просто определяться функциями (иначе говоря, быть [feature-driven](#)), где требования или группы требований можно будет однозначно связать с главными функциями (или фичами) системы.

- Алгоритмическая лента будет приоритизировать твиты от премиум-пользователей;
- Каждый твит будет иметь сгенерированный уникальный идентификатор размером 64 бита.

Интеграционные — функциональные требования охватывают те внешние сервисы и функции, с которыми наша система должна обязательно быть связана.

В качестве примера, вот как могло бы выглядеть описание интеграции приложений Нетфликса с умными колонками:

Как авторизованный пользователь мобильного приложения Netflix с активной подпиской, я могу запустить воспроизведение сериала из библиотеки с помощью голосовой команды для умной колонки.

Или перспективное улучшение для Telegram:

Как авторизованный пользователь мобильного приложения Telegram, я могу загрузить отправленное мной видеосообщение как видео в TikTok, с помощью опции в контекстном меню

сообщения.

Сбор, обработка и хранение данных. Требования, связанные с данными — важная дополнительная область, определяющая как и какие данные нам будет необходимо собирать для того, чтобы обеспечивать работу функций системы и её дальнейшее развитие.

- Приложение водителя такси каждые 2 секунды отправляет на сервер текущие показатели датчиков движения смартфона, на котором установлено приложение.

В качестве довольно стандартного функционального требования такого рода приведём такое:

Веб-приложение my-example.com должно отправлять метрики по событиям «**клик на баннер X**» и «**старт встроенного медиа Y**» в систему Google Analytics.

Функциональные требования — это возможности, которые система предоставляет пользователю.

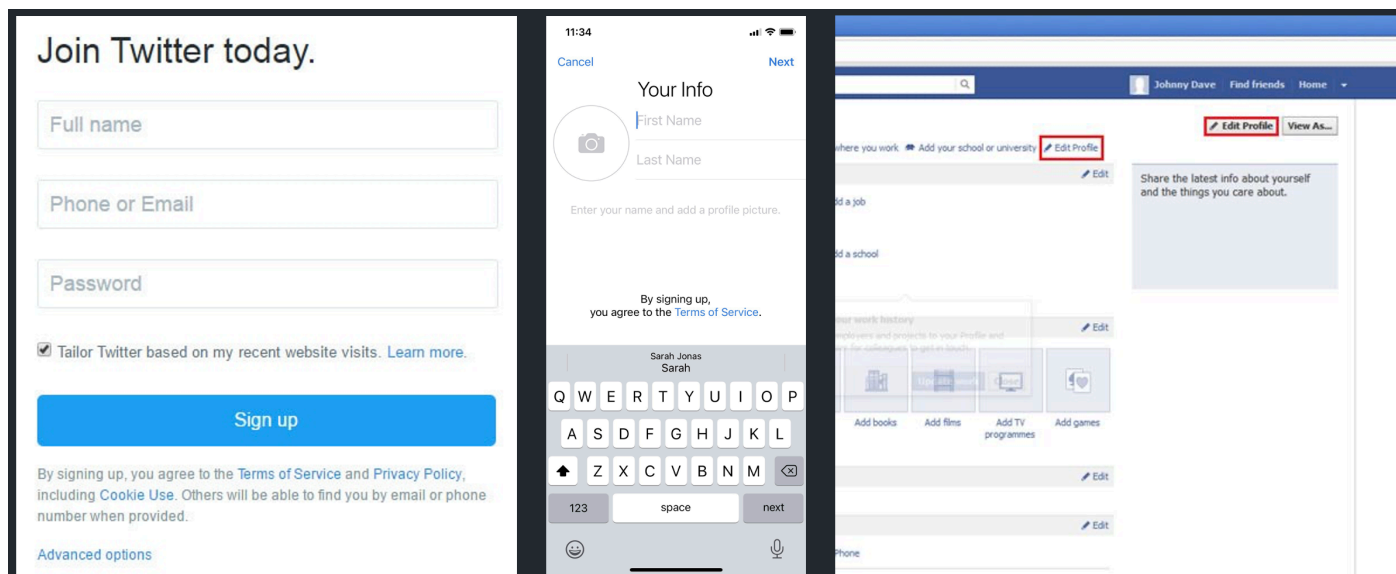
В процессе обсуждения рекомендуем двигаться итерациями и останавливаться на нескольких возможностях для реализации. Например, для социальных сетей это:

- добавление в друзья;
- общение между пользователями;
- создание пабликов.

Для этого стоит выписать наиболее известные свойства системы в порядке важности и провести черту под теми, над которыми мы планируем сконцентрироваться в дальнейшем.

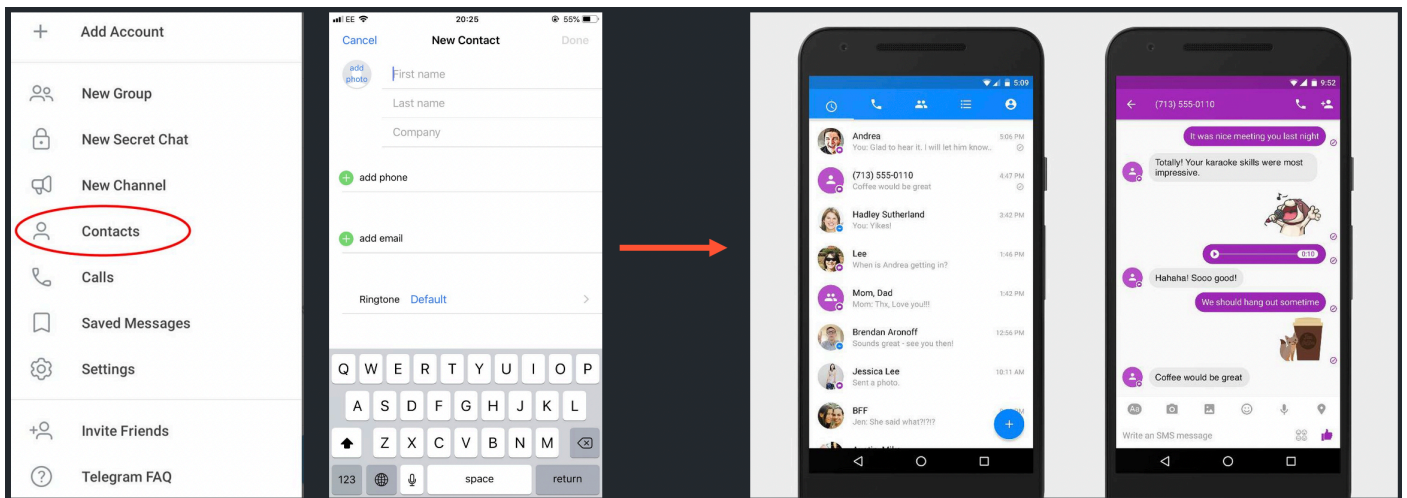
Общее для социальных сетей

- создание профиля (добавление описания и фото).

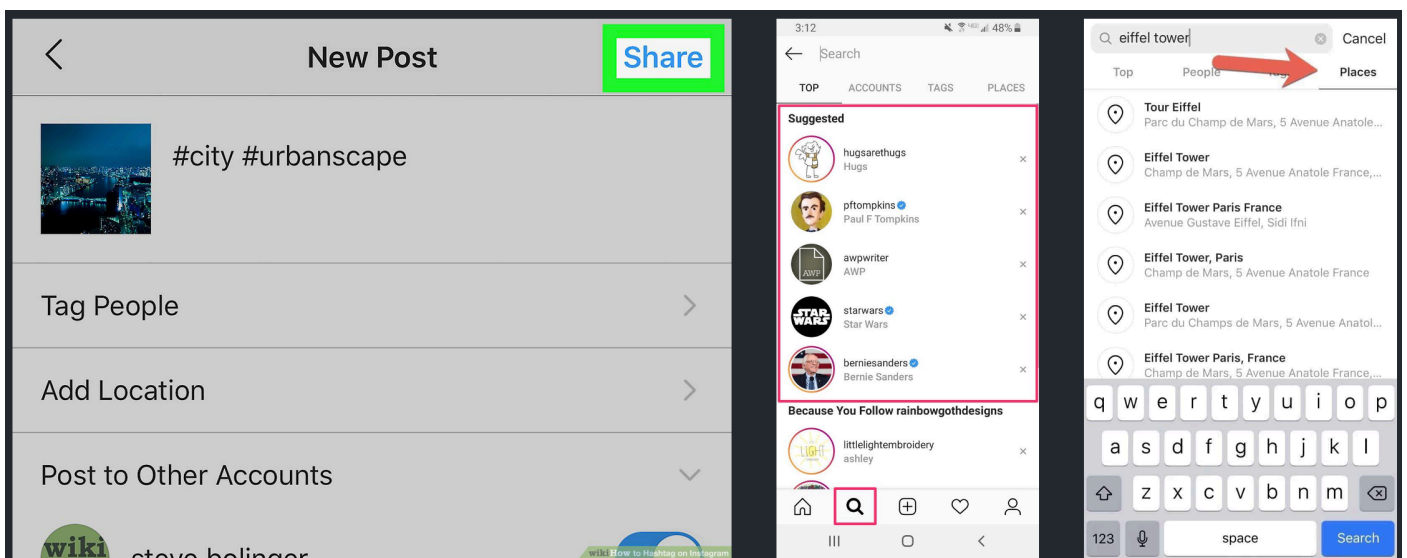


Затем, в зависимости от направленности социальной сети, можем представить такие функции, как:

- добавление контактов из сети (по номеру телефона?);
- общение со своими контактами (только текстом или с голосовыми и видео?).



Какие-то соцсети могут поддерживать все направления и дополнительные возможности, вроде добавления геотегов, поиска контента, рекомендаций и т.п.



Основные мысли для создания социальной сети

Имеет смысл выбрать тот функционал, на котором остановимся:

- создание профиля с юзернеймом;
- создание только текстовых постов;
- подписка на посты других пользователей.

А дальше продолжать двигаться итерациями и обсуждать следующий набор:

- добавление контактов по номеру телефона;

- общение со своими контактами с видеозвонками;
- добавление тегов и геометок к постам;
- полнотекстовый поиск контента;
- отображение постов от пользователей поблизости и др.

> Нефункциональные требования

Технические ограничения

Одной важной и не совсем очевидной разновидностью нефункциональных требований могут являться технические ограничения на систему, про которые тоже стоит подумать на старте. Например, предпочтительный стек технологий, выбор типа архитектуры или возможность использовать облачные решения. Так, следующие требования тоже будут являться нефункциональными:

- Сервисы системы должны быть реализованы в парадигме бессерверной обработки данных;
- Сервисы системы должны быть спроектированы в условиях гибридной инфраструктуры. Хранение и обработка данных — на локальных мощностях (on-premise), сервисы не хранящие в себе состояние — могут быть развёрнуты в облаке.

Качественные требования

Такие виды требований ещё часто называют «[ilities](#)» (от английского окончания качественных прилагательных — scalability, maintainability, etc.) или *****quality of service**. В этом случае, перебрать все [десятки видов доменных областей качественных требований](#) вряд ли представится возможным, разве что если вы не инженер в Boeing, но несколько самых характерных для программных продуктов и сервисов рассмотрим ниже:

Производительность и масштабируемость

При расчёте [нагрузки на систему для веб-приложений](#) обычно используются такие метрики, как Requests per second (RPS), Connections per second (CPS), Queries per second (QPS), Throughput и Latency.

Поэтому, вместо того, чтобы в случае сервиса такси просто потребовать: *****сервис пуш-уведомлений должен выдерживать высокую пиковую нагрузку** или *****система должна выдерживать большое количество активных поездок**, правильно будет сформулировать это так:

- Система должна выдерживать пиковую нагрузку в 200 000 CPS для активных поездок в каждой из географических точек присутствия инфраструктуры;

- Подсистема рассылки пуш-уведомлений должна выдерживать пиковую нагрузку в 500 000 RPS.

Отзывчивость

Интуитивное требование о том, что части системы, с которыми взаимодействует юзер, должны отвечать быстро и быть отзывчивыми, тоже на самом деле можно хорошо формализовать. Для этого можно обратиться к различным метрикам производительности UI. Например, это могут быть [First Contentful Paint \(FCP\)](#) и [Largest Contentful Paint \(LCP\)](#), определяющие, насколько быстро загружаются первый элемент и самый большой текстовый блок или медиа на веб-страничке, соответственно. Ещё одна популярная метрика — [Time To Interactive \(TTI\)](#) - насколько быстро становятся активными все элементы, с которыми может взаимодействовать пользователь.

Например, требование о том, что лента твиттера загружается быстро для большинства пользователей можно было бы записать вот так.

- TTI начальной загрузки веб-приложения x.com/home для авторизованных пользователей должен составлять 2.5 секунд для 90-го перцентиля.

Для видеоконтента и медиа могут быть важны метрики [Time To First Byte \(TTFB\)](#), а так же, например [Bit rate, Buffer и Lag](#)

Например, требование о том, что пользователи не ждут долго загрузки видео в ленте можно формализовать вот так:

- TTFB для встроенного медиаплеера в ленте x.com/home составляет 50 мс для 95-го перцентиля пользователей в североамериканском регионе.

Можно также изучить актуальный репорт [pagespeed.web.dev](#) для x.com/home чтобы познакомиться с остальными метриками и лучше понять как их интерпретировать.

Надёжность и доступность

Чтобы также в рамках метрик поговорить о том, что система должна быть доступной и пользователи не должны видеть слишком много ошибок, существуют метрики из семейства [Service Level Agreement \(SLA\)](#), обычно определяющие публичные гарантии, которые система предоставляет своим пользователям.

Скорее всего, самыми распространенными из таких будут [uptime](#) (процент времени, который за контрольный период система была готова к ответу и функционировала штатно) и [availability](#) (вероятность, с которой система была готова к ответу и функционировала штатно для конечных пользователей за контрольный период времени).

Например, то, что в сервисе такси функция вызова такси должна работать с высокой надёжностью, можно было бы описать вот так:

- Функция вызова такси через приложение пассажира должна гарантировать SLA доступности в 99.99% за годовой период измерений.

А в качестве хорошей иллюстрации того, как в реальном мире считаются uptime метрики, можно зайти на <https://status.openai.com/>, отображающей метрики текущей доступности API для ChatGPT. На момент написания этих материалов, аптайм составлял 98.77%, что в грубом пересчёте можно, например, интерпретировать, как 4.5 дня простоя за год. То есть, с точки зрения дерзкого стартапа, решившего сделать киллер-приложение на основе новой ChatGPT, нужно планировать из её публичного SLA, что без всяких гарантий и предупреждения, всё может перестать работать на несколько часов или даже дней.

Если же хотим подчеркнуть надёжность системы в плане работы с ошибками и отказоустойчивости, можно обратиться к классическим способам измерения надёжности инженерных систем — [Mean time between failures\(MTBF\)](#) или [Mean Time To Repair\(MTTR\)](#)

Безопасность

Последний важный домен требований — это безопасность. Много примеров того, как будут выглядеть требования из этой области, можно найти в современных стандартах, формализующих необходимые практики в виде готовых чеклистов:

- [OWASP Application Security Verification Standard](#)
- [AWS Security Checklist](#)
- [Center for Internet Security benchmarks](#)

Например, вот такое требование по безопасности уже вполне соответствует критериям измеримости (хотя, на самом деле, и содержит в себе десятки вложенных дочерних требований из самого стандарта, некоторые из которых могут уже быть функциональными).

Все экземпляры реляционной базы данных для хранения пользовательских профилей должны соответствовать стандартам безопасности для Postgres 14, определенным в CIS PostgreSQL 14 benchmark.

В качестве более конкретных технических нефункциональных требований в области безопасности можно было бы привести такие:

- Все хранимые данные должны быть зашифрованы с помощью централизованно-управляемых ключей;
- Любой доступ к внутренним сервисам системы должен быть аутентифицирован.

Нефункциональные требования — это то, как должна вести себя система в работе.

Типичные вопросы, на которые хотим ответить:

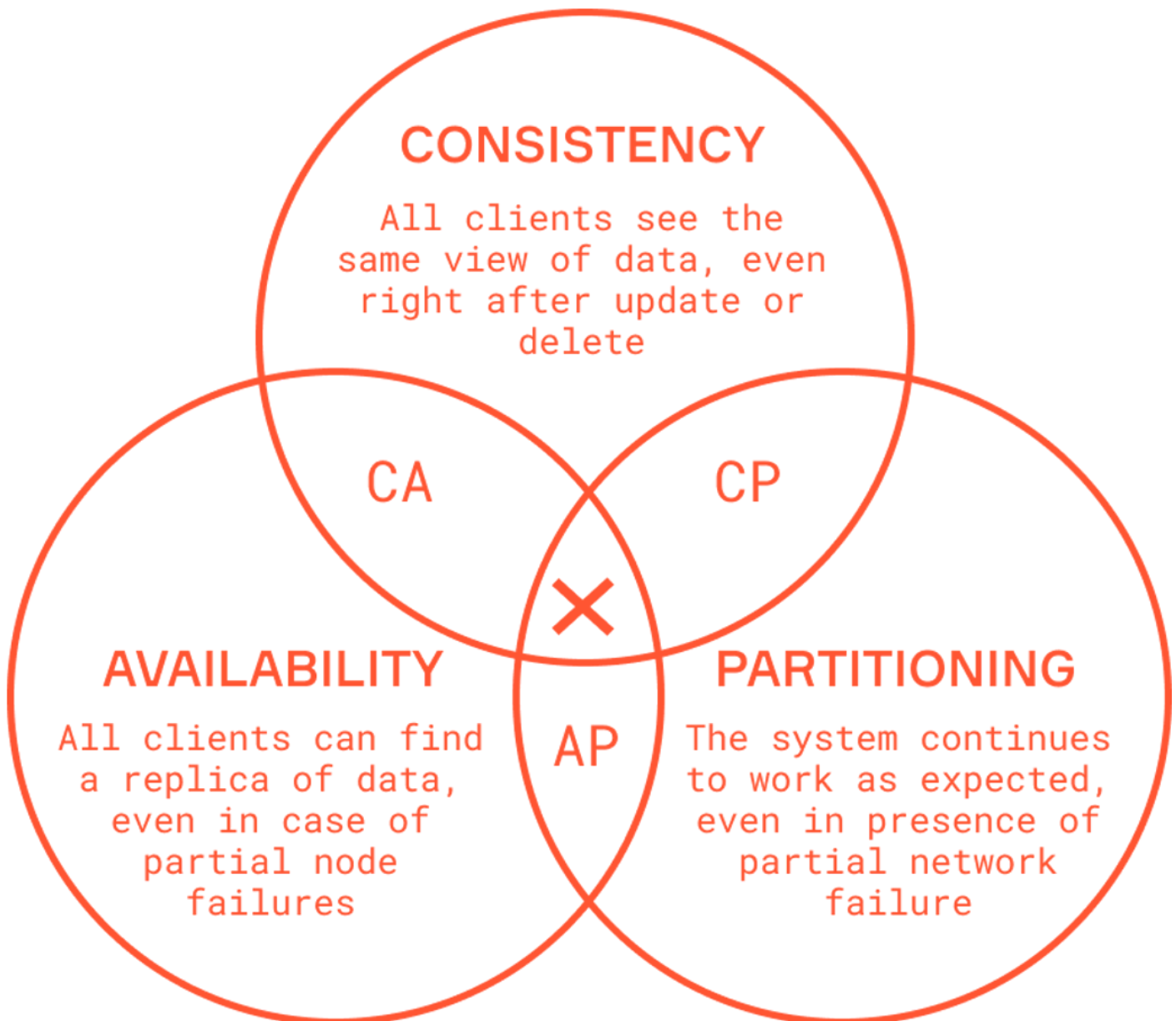
- Должна ли система быть абсолютно надёжной или какие-то данные могут теряться?
- Что нам важнее — быстрый ответ системы или гарантия учёта последних изменений?
- С какой задержкой могут применяться вносимые нами изменения?

- С какой задержкой отображаются запрашиваемые данные?

Changes in the Admin console or to Google services

Changing settings or performing tasks, such as managing mobile devices and users, are not always immediately reflected. Some changes can take up to 24 hours to take effect. Sometimes, you'll see a warning message if there's an expected delay.

> CAP теорема



С первого взгляда все эти вопросы могут показаться странными — почему бы не сделать систему, которая удовлетворяет вообще всем требованиям? Но есть чисто теоретические ограничения, почему так не может быть, поэтому приходится всегда чем-то жертвовать.

Например, CAP теорема говорит нам о том, что нельзя создать систему, одновременно всегда доступную, консистентную и устойчивую к разделению на части (например, по сети).

В зависимости от ситуации мы делаем выбор в пользу того или иного ограничения.

Для социальной сети важно, чтобы она всегда была доступной. Социальная сеть также разделена, имея пользователей по всему миру (и сервера, соответственно). Однако, если пользователь публикует пост, возможно, мы не сразу его увидим. Ничего страшного, если произойдёт задержка, рано или поздно он всё равно станет доступен для всех пользователей. Получается, что, консистентность не так критична.

twitter-archive/twitter-kit-android

#107 Initial loading of Tweet really slow



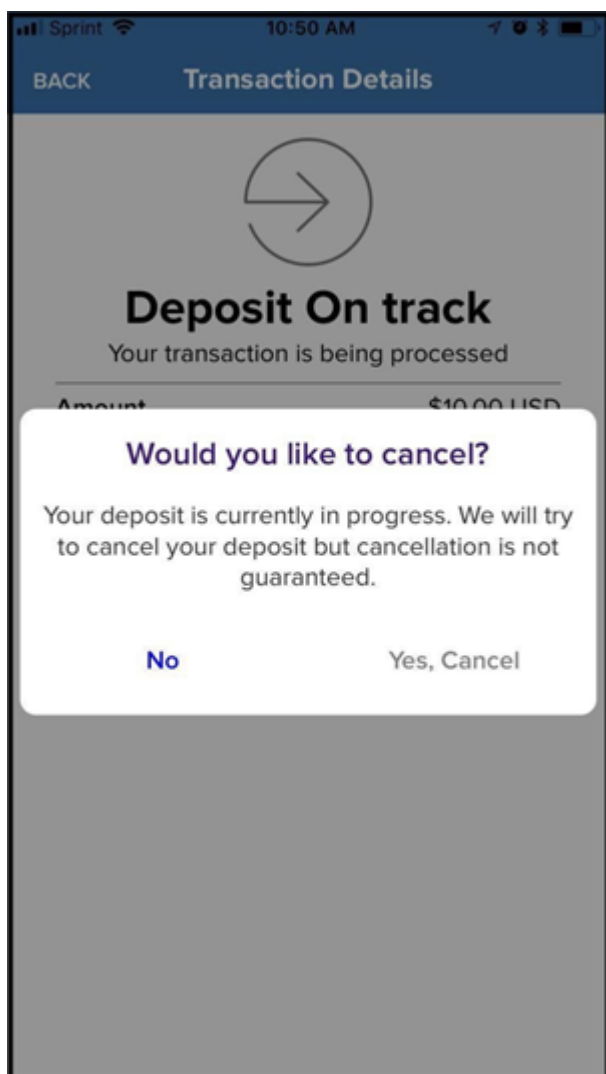
 2 comments



Sammekl opened on February 9, 2018



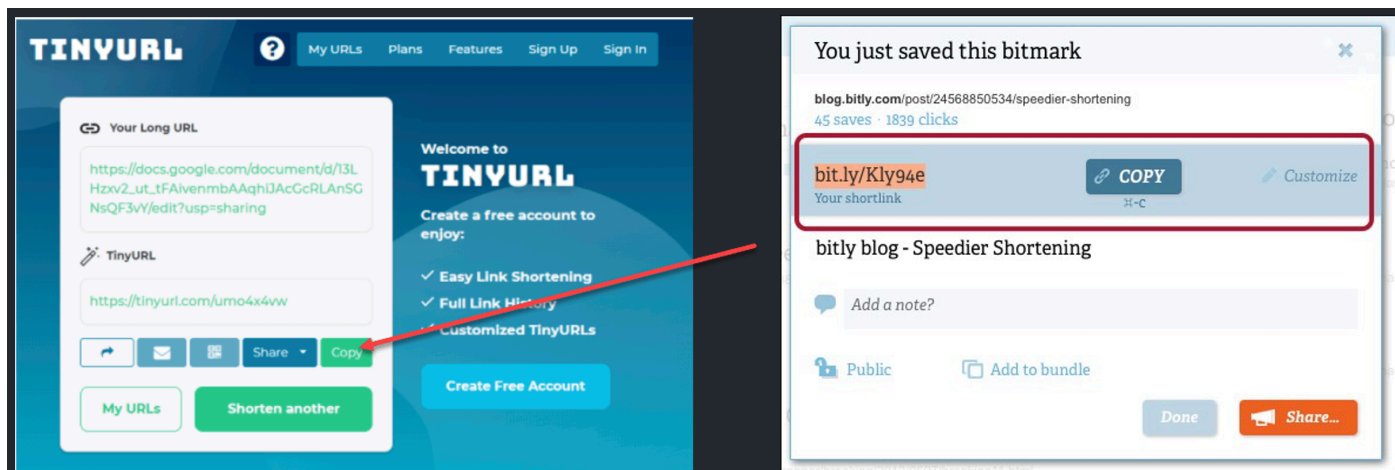
В случае с банковским приложением успешно проведенная денежная транзакция должна означать поступление и видимость средств на счету адресата, никак иначе. Ценой этому может быть недоступность приложения какое-то время.



> Сокращатель ссылок

Зачем нужен данный сервис?

Чтобы превращать длинные ссылки в короткие и делиться ими.



Функциональные требования

- По полной ссылке пользователь может сгенерировать короткую;
- Когда пользователь переходит по короткой ссылке, он перенаправляется на исходную;
- У пользователей есть возможность самим выбрать сокращённый синоним;
- Ссылки «протухают» через какое-то время. Пользователи могут его изменять.

Нефункциональные требования

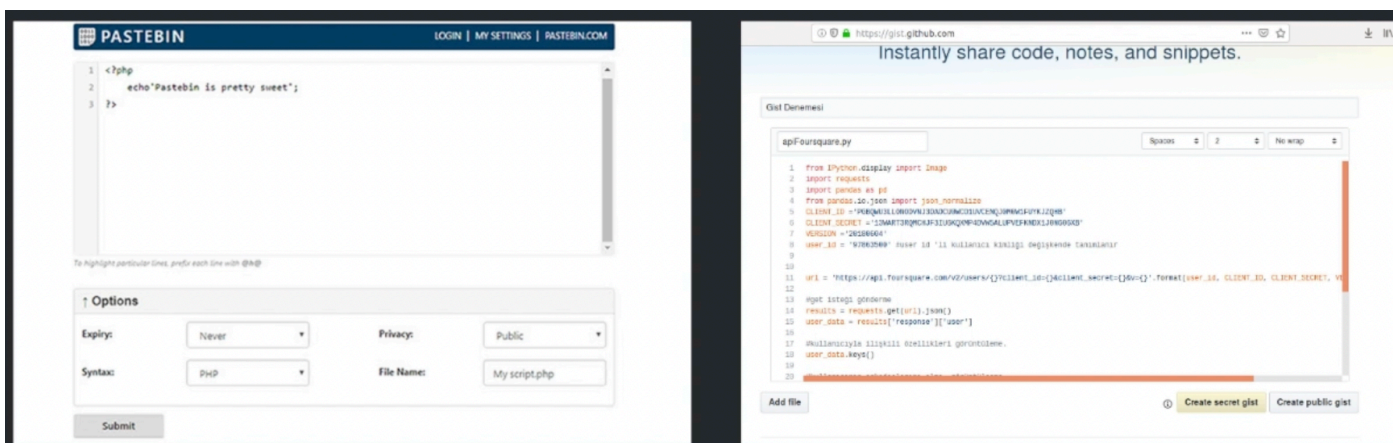
- Высокая доступность сервиса – иначе ссылки просто перестают работать.
- Перенаправление должно происходить с минимальной задержкой.
- Короткие ссылки должны быть случайны настолько, чтобы их нельзя было подобрать.

Бонус: сбор аналитики по переходам, предоставление API для разработчиков.

> Хостинг текстов

Зачем нужен такой сервис?

Чтобы делиться кусками кода и искать в нем баги вместе.



Функциональные требования

- Пользователи могут загрузить отрывок текста и получить URL для доступа к нему;
- Пользователи могут загружать только текстовые данные;

- У пользователей есть возможность выбрать синоним для доступа;
 - Ссылки «протухают» через какое-то время. Пользователи могут его изменять.
-

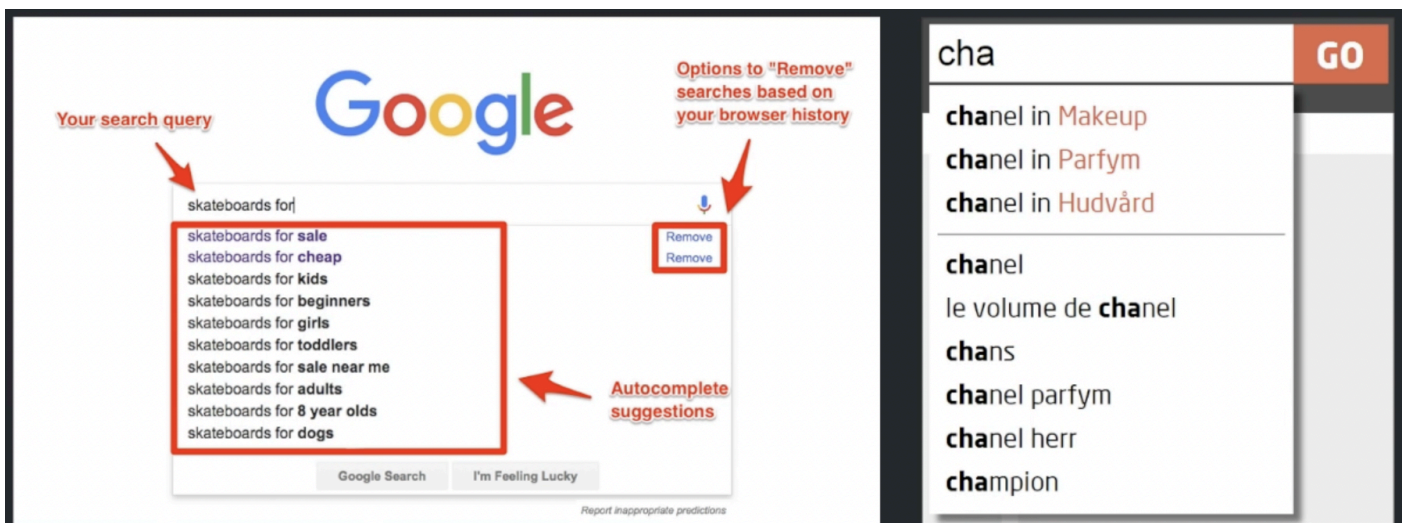
Нефункциональные требования

- Высокая надёжность сервиса — загруженные данные не должны исчезать;
- Высокая доступность сервиса — иначе доступ к нужным данным пропадает;
- Сохранённые тексты должны отображаться с минимальной задержкой;
- Короткие ссылки должны быть случайны настолько, чтобы их нельзя было подобрать.

> Автодополнение

Зачем нужен такой сервис?

Чтобы при вводе первых букв запроса предлагались варианты дополнения. Например, при поиске «Н» сразу дополнялось до «НФТ купить».



Функциональные требования

- Пользователи вбивают запрос, для него предлагается топ-5 дополнений;
 - Варианты обновляются по мере вбивания букв и слов;
 - Пользователи могут видеть в подсказках предыдущие релевантные результаты.
-

Нефункциональные требования

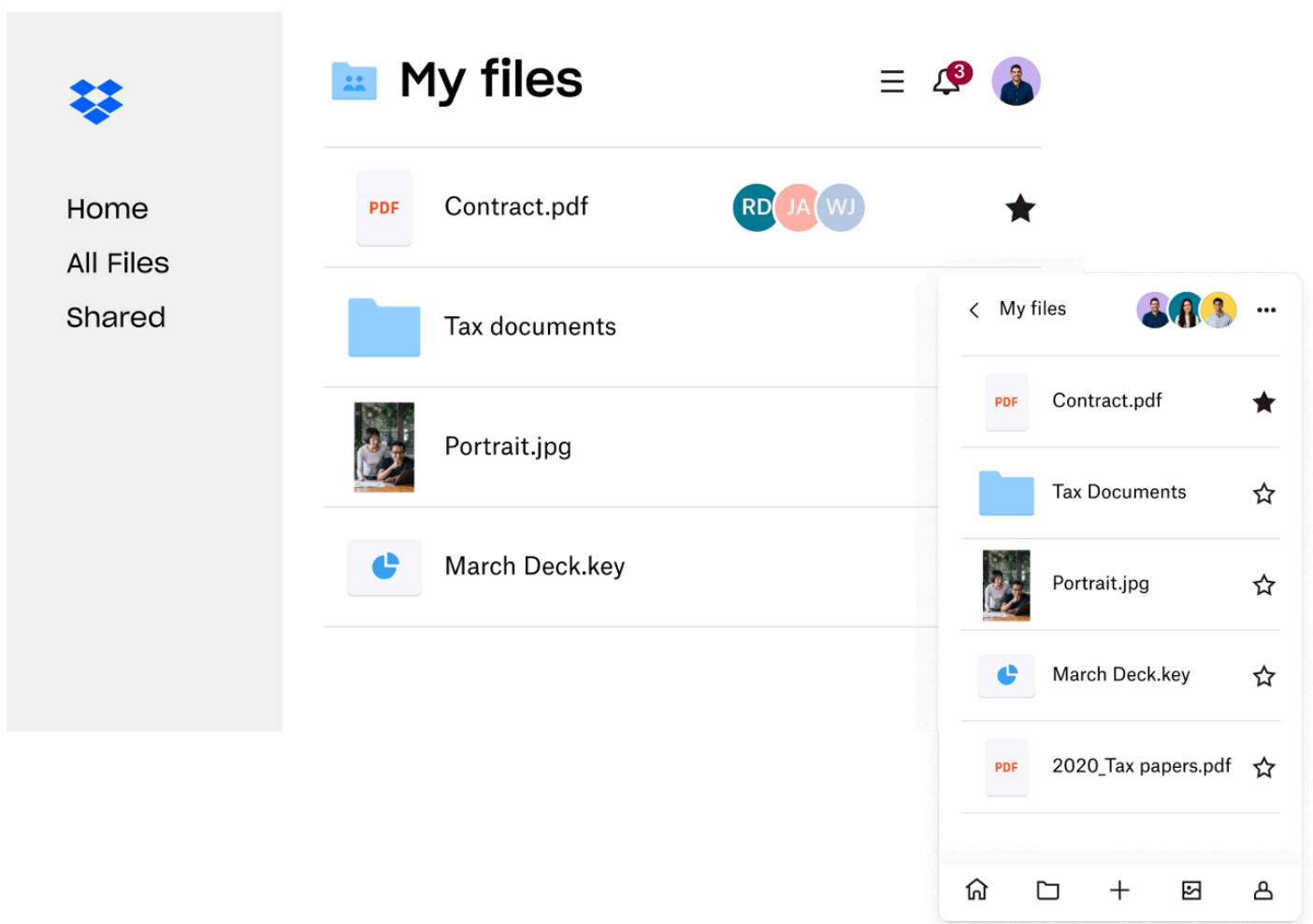
- Высокая отзывчивость сервиса – дополнения обновляются почти в реальном времени.

Бонус: учитывать прошлую историю запросов, профиль пользователя, контекст.

> Облачный диск

Зачем нужен такой сервис?

Чтобы файлы были везде доступны.



Функциональные требования

- Пользователи могут загружать и скачивать свои файлы с любого устройства;
- Пользователи могут делиться файлами и директориями с другими;
- Между всеми устройствами происходить автоматическая синхронизация;

- Можно редактировать файл офлайн, синхронизация произойдет при выходе в сеть.
-

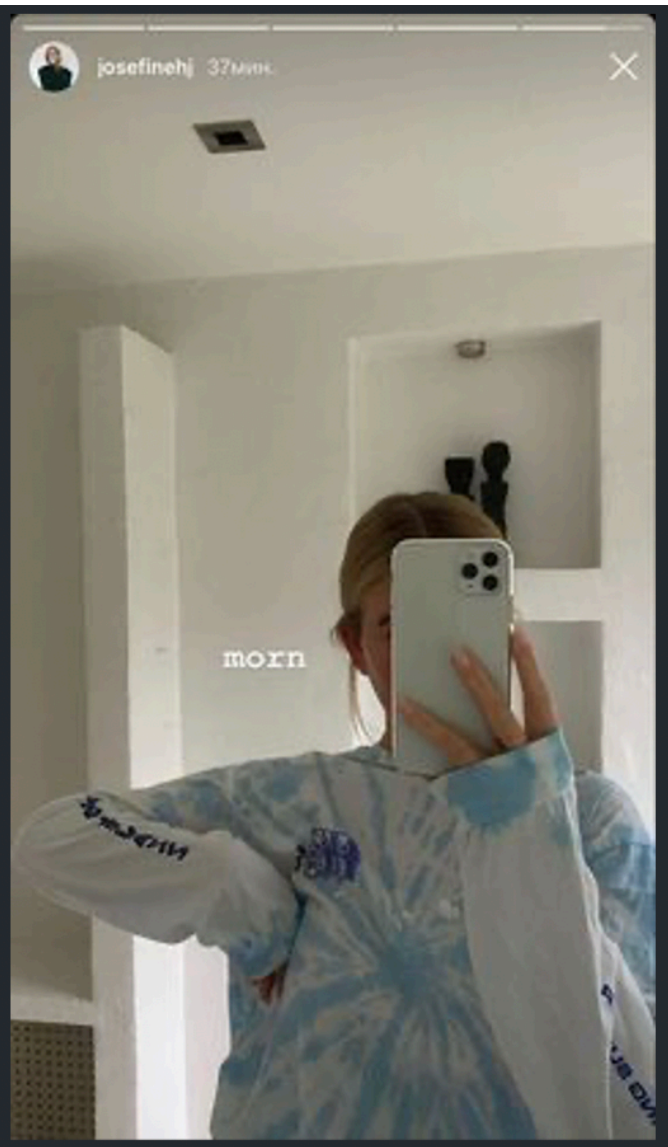
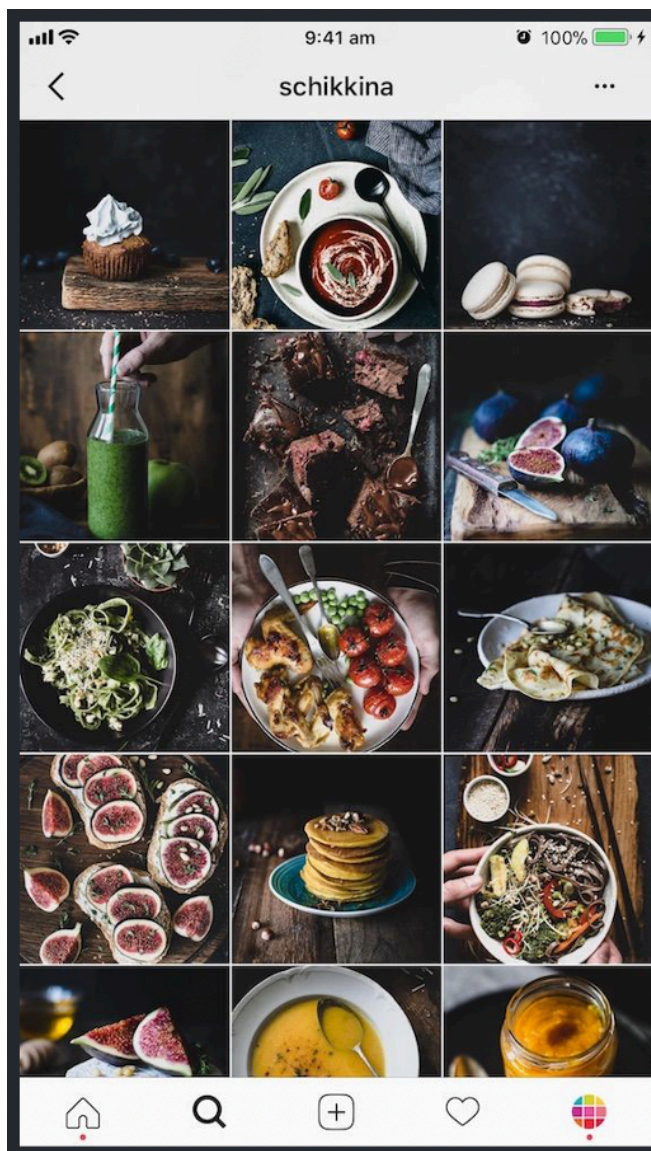
Нефункциональные требования

- (A) Атомарность — файл либо обновляется, либо нет, но не становится «битым»;
- (C) Консистентность — при любом сценарии работы данные в облаке всегда валидны;
- (I) Изоляция — можно править разные файлы с разных устройств, всё будет хорошо;
- (D) Устойчивость — после зелёной галочки на файле кофе уже можно выливать на ноутбук.

> Приложение для выкладывания фотографий

Зачем нужен такой сервис?

Чтобы делиться фотографиями в зеркале и с едой с друзьями.



Функциональные требования

- Пользователи могут загружать, скачивать и просматривать фотографии;
- Пользователи могут искать другие фотографии по описанию;
- Пользователи могут подписываться на фотографии друг друга;
- Есть лента из популярных фото всех отслеживаемых пользователей.

Нефункциональные требования

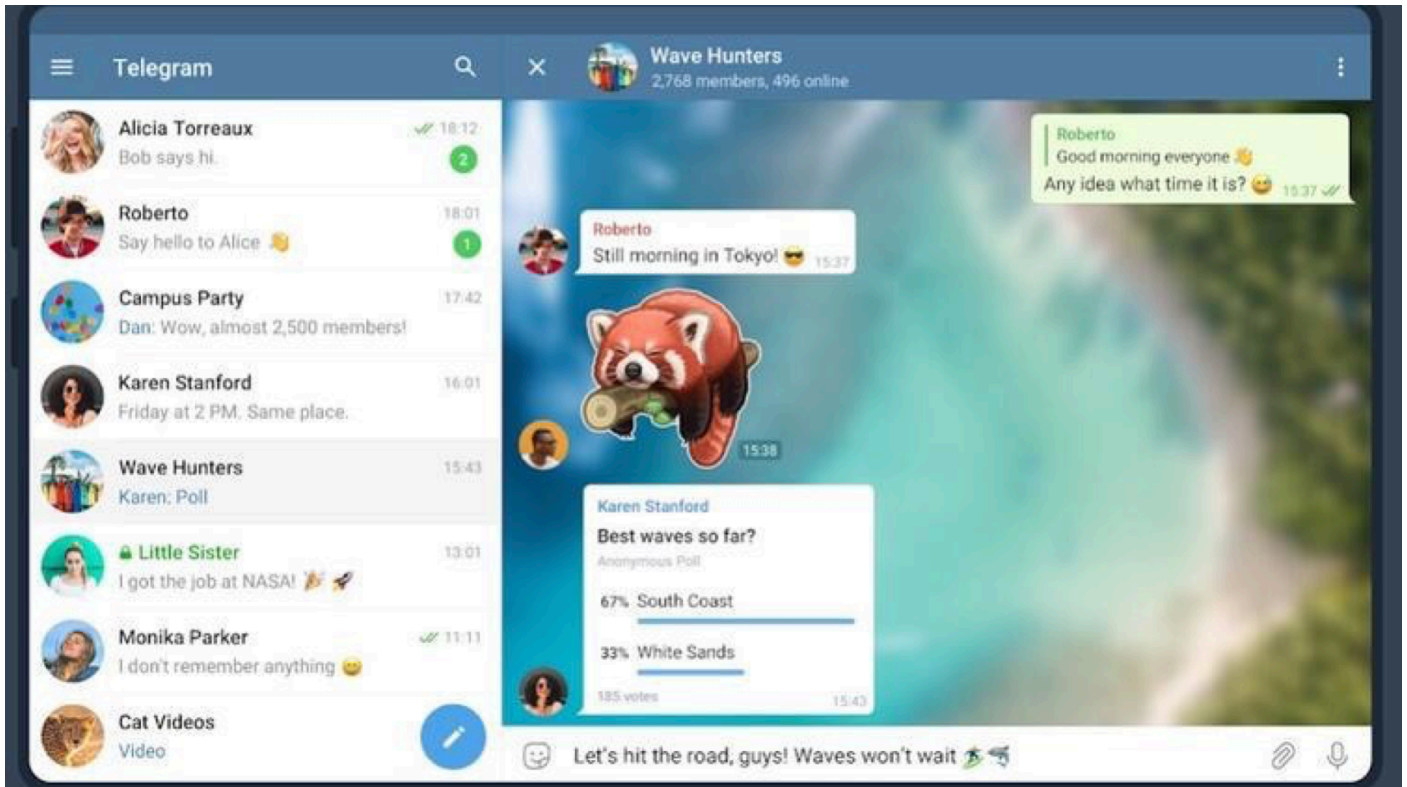
- Высокая доступность сервиса – иначе пользователи перейдут в Snapchat или TikTok.
- Лента с фотографиями должна отображаться за сотни миллисекунд (иначе см п. 1).
- Консистентность не так критична – подписчики сколько-то потерпят без ваших селфи.

- Высокая надёжность сервиса — ни одно селфи или фото еды не должно потеряться.

> Телеграм

Зачем нужен такой сервис?

Чтобы от общения вас отвлекали с работы, где не купили Slack.



Функциональные требования

- Можно общаться один на один в чатах с другими пользователями;
- Можно понять, онлайн ли пользователь и когда был последний раз;
- Сервис обеспечивает постоянное хранение всей истории переписки.

Нефункциональные требования

- Общение происходит в реальном времени с минимальными задержками;
- Консистентность — одинаковые сообщения в чатах на всех устройствах;
- Высокая доступность сервиса — иначе пользователи перейдут в там-там.

Бонус: стикеры, групповые чаты, кружочки с видео, беседы как в Clubhouse, стена, истории.

> Твиттер

Зачем нужен такой сервис?

Чтобы делиться короткими сообщениями со всеми.



Функциональные требования

- Пользователи могут добавлять короткие посты;
- Посты могут содержать фото или видео;
- Пользователи могут отслеживать посты других;
- Есть лента из твитов всех отслеживаемых пользователей.

Нефункциональные требования

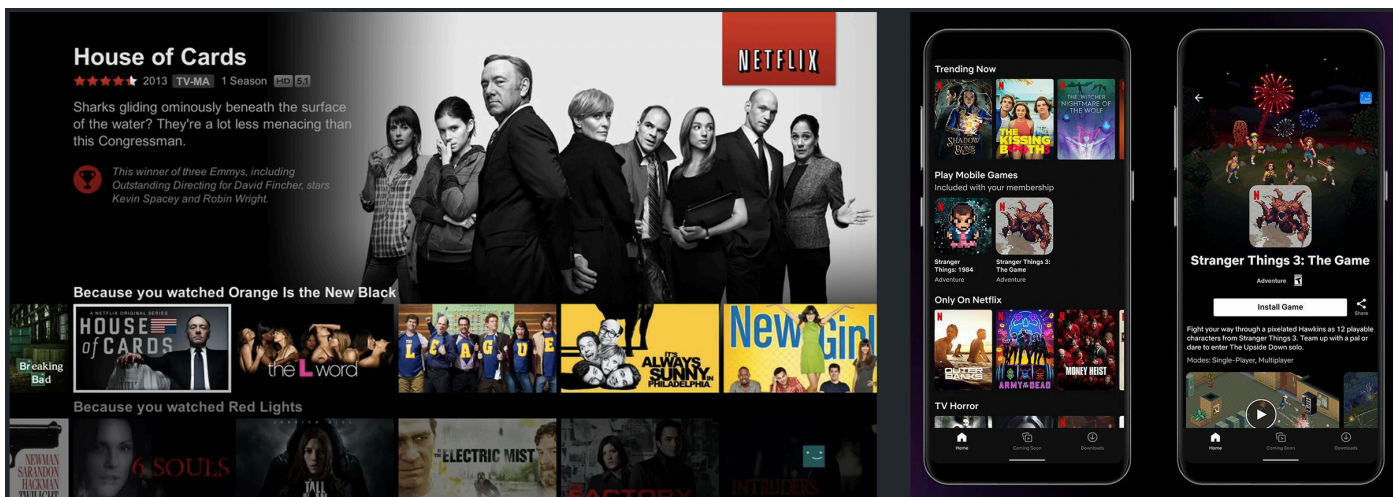
- Высокая доступность сервиса — иначе пользователи вернутся в живой журнал;
- Лента с твитами должна отображаться за сотни миллисекунд (иначе см п. 1);
- Консистентность не так критична — подписчики сколько-то потерпят без ваших мыслей.

Бонус: поиск, реплаи, горячие темы, уведомления, рекомендации, аватарки с NFT обезьянами.

> Нетфликс

Зачем нужен такой сервис?

Чтобы смотреть любимые фильмы, сериалы и передачи.



Функциональные требования

- Пользователи могут просматривать фильмы и сериалы;
- Пользователи могут искать контент по названию;
- Сервис отслеживает оценки и статистику просмотров;
- Пользователям рекомендуются новые фильмы и сериалы.

Нефункциональные требования

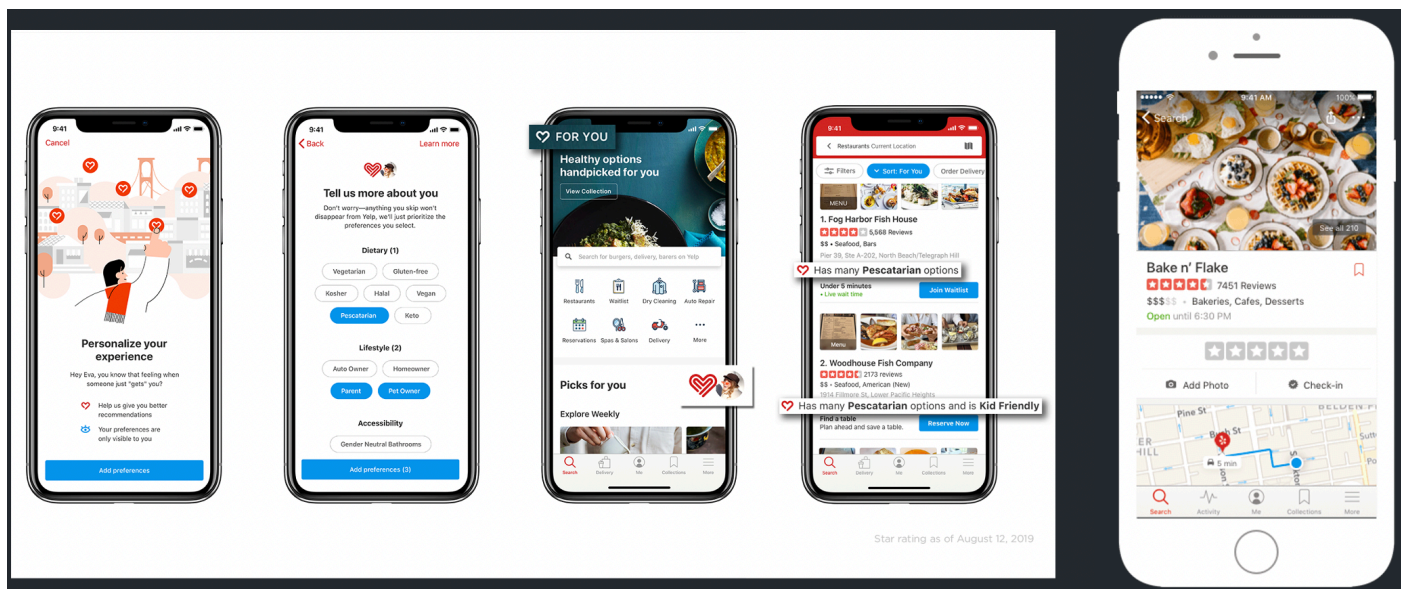
- Высокая доступность сервиса — иначе пользователи будут смотреть котиков у конкурентов;
- Высокая отзывчивость сервиса — видео должны проигрываться без подвисаний;
- Консистентность не так критична — после недельного ожидания одна минута не заметна.

Бонус: жанры, популярное, избранное, списки на посмотреть потом.

> Поиск ресторанов

Зачем нужен такой сервис?

Чтобы поесть не холодную еду из зелёных и жёлтых сумок.



Функциональные требования

- Пользователи могут добавлять/обновлять/удалять заведения;
- Пользователи могут находить подходящие заведения поблизости;
- Пользователи могут оставлять отзывы к посещённым местам с фото и оценками.

Нефункциональные требования

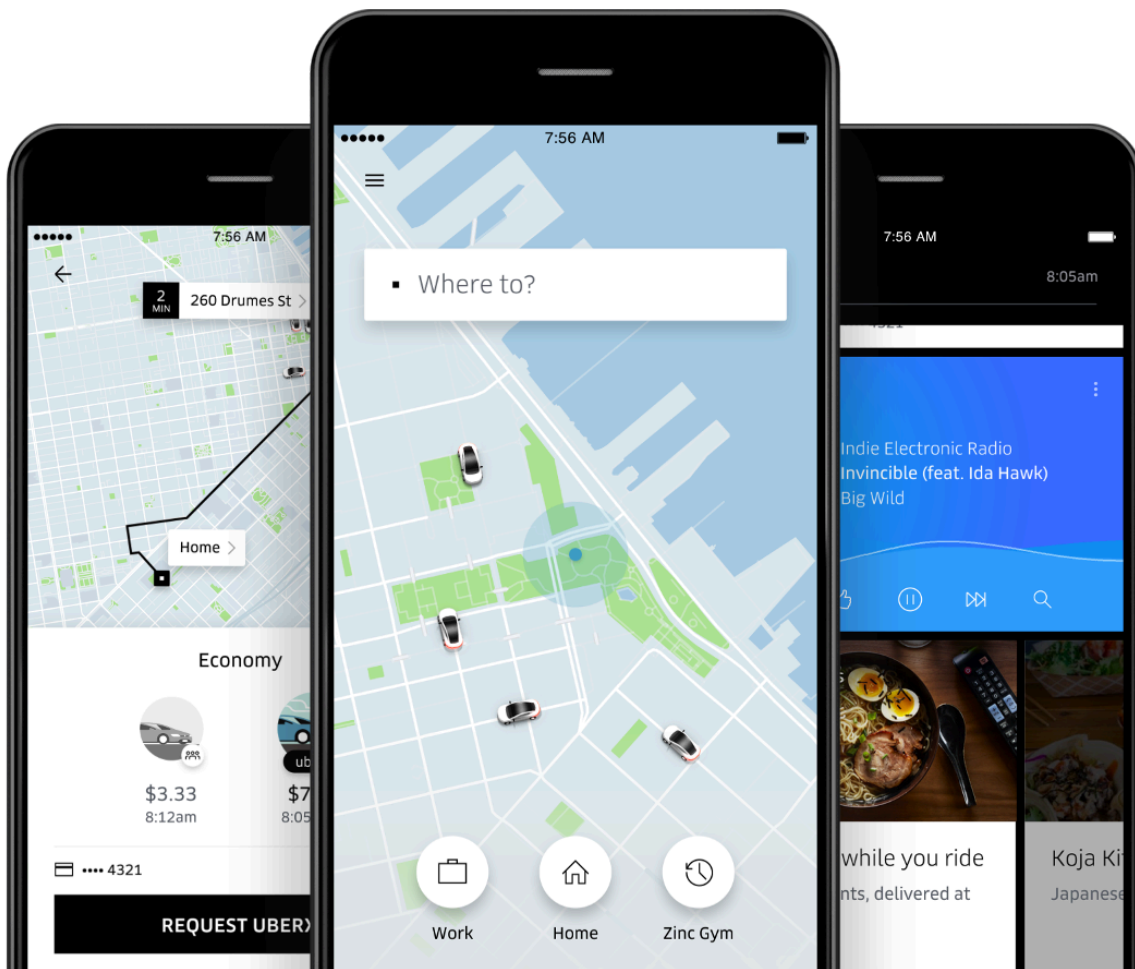
- Поиск должен происходить быстро, иначе пользователь закажет доставку;
- Гораздо большая нагрузка ожидается на подсервис поиска, чем на редактирование.

Бонус: фильтры по категориям, рекомендации новых заведений.

> Сервис такси

Зачем нужен такой сервис?

Чтобы легко доехать из точки А в точку Б с водителем.



Функциональные требования

- Водители могут выходить на смену и ждать заказ;
- Пассажиры могут находить водителей поблизости;
- Пассажиры могут заказывать поездку, для неё находится водитель;
- Позиции отслеживаются с момента принятия поездки и до её окончания. В конце поездки водитель продолжает смену.

Нефункциональные требования

- Высокая доступность сервиса — иначе пассажиры поедут на метро, а водители на вокзал;
- Низкое время ожидания для пассажира, маленький простой для водителя.

Бонус: оценки водителям, оценки пассажирам, объяснение ценообразования.

> Заключение

На этой лекции:

- научились собирать требования для дизайна сервисов;
- делить их на функциональные и нефункциональные;
- выделять основные из общего перечня.

> Дополнительное чтение

Глава 1: “Software Requirements” из Software Engineering Body of Knowledge (SWEBOK) v4 2022
([ссылка на версию v4 2022 года из публичного превью](#))

Больше о метриках в вопросах надежности:

- <https://sre.google/sre-book/service-level-objectives/>
- <https://sre.google/sre-book/monitoring-distributed-systems/>

[О формулировках нефункциональных требований](#)

[Метрики качества доставки видео](#)

[О видах требований к консистентности и метриках на примере Azure Cosmos DB](#)